

Scripting Language: Lab Questions

Lab 1: Interactive Registration Form (Client-Side Focus)

Concepts Covered: JavaScript, DOM, Events, Forms, Regular Expressions, Client-Side Validation.

Problem Statement: Create an HTML registration form for a "Tech Conference" with fields for Name, Email, Phone Number, Password, and Confirm Password. Write a JavaScript script to handle the following requirements **without** submitting the form to a server yet:

1. **Real-time Validation:** As the user types, validate the **Email** using a Regular Expression. If valid, the border of the input box turns green; if invalid, it turns red.
 2. **Password Logic:** Ensure the password is at least 8 characters long and contains at least one number.
 3. **Form Submission Prevention:** On clicking "Register", check if "Password" and "Confirm Password" match. If they do not, display an error message using an `alert()` and prevent the form submission.
 4. If all validations pass, dynamically create a new `alert` that displays a welcome message: *"Thank you [Name], you have successfully registered!"*
-

Lab 2: The Shopping Cart Session (Basic PHP & State Management)

Concepts Covered: PHP Syntax, Arrays, `$_POST`, Sessions, Cookies, Control Structures.

Problem Statement: Create a multi-page PHP application to simulate a simple shopping cart.

1. **Page 1 (Products):** Display an associative array of 5 products (Name, Price) using a `foreach` loop. Next to each product, add an "Add to Cart" button.
2. **Logic:** When a user clicks "Add to Cart", use `$_POST` to send the data to a script that stores the selected item in a **PHP Session**.
3. **Page 2 (Cart):** Create a separate page called **cart.php**. This page should check if the Session variable exists.
 - If items exist, display them in a table and calculate the total price using a loop.
 - If the cart is empty, display "Your cart is empty."

4. **Cookies:** Set a cookie named `last_visit` that stores the current timestamp. Display "You last visited on: [Date]" at the bottom of the page

Lab 3: Employee Management System (Database Connectivity)

Concepts Covered: MySQL Connect, INSERT, SELECT, UPDATE, Aggregate Functions, SQL Clauses.

Problem Statement: Develop a PHP/MySQL application to manage employee records.

1. **Database Setup:** Create a MySQL table `employees` with columns: `id`, `name`, `department`, `salary`, and `joining_date`.
 2. **Input Form:** Create a form to insert a new employee. Use PHP to `INSERT` this data into the database.
 3. **Reporting:** Create a page that runs a `SELECT` query to display all employees in an HTML table.
 - **Filter:** The query must use `ORDER BY` to sort employees by `salary` (descending).
-

Lab 4: Banking System Class Hierarchy (Advanced PHP / OOP)

Concepts Covered: Classes, Objects, Inheritance, Encapsulation, Constructors, Method Overriding, Exception Handling.

Problem Statement: Write a PHP script using Object Oriented Programming to model a Bank Account system.

1. **Base Class:** Define a class `BankAccount` with private properties `accountNumber` and `balance`. Include a Constructor to initialize these.
 - Add methods for `deposit($amount)` and `withdraw($amount)`.
 - Use Exception Handling: Throw an exception if the withdrawal amount is greater than the balance.
2. **Derived Class:** Create a class `SavingsAccount` that inherits from `BankAccount`.
 - Add a property `interestRate`.
 - Override the `deposit` method to add a 2% bonus if the deposit is over \$1000.
3. **Execution:** Instantiate a `SavingsAccount` object, perform a few transactions (including one that triggers an exception), and print the final balance.

Lab 5: Live Search Portal (AJAX & Full Integration)

Concepts Covered: AJAX, PHP + MySQL, DOM, JavaScript XMLHttpRequest or fetch.

Problem Statement: Create a "Live Search" feature for a library database without reloading the page.

1. **Database:** Use a MySQL table books with title and author.
 2. **Frontend:** Create a simple HTML page with a text input box labeled "Search for a book...".
 3. **AJAX Logic:** Write a JavaScript function that triggers on the keyup event.
 - It should send the current input value to a PHP script (search.php) in the background using AJAX.
 4. **Backend:** The search.php file should receive the query, perform a SELECT ... LIKE '%...%' query on the MySQL database, and return the results.
 5. **Result:** Display the returned book titles in a list under the search box dynamically.
-

Instructions for Lab Report Submission

For each lab question, your report must follow this exact format:

- **Objective:** A one-line statement of what the program does.
- **Concept Description:** A brief paragraph (50-100 words) explaining the core technology used (e.g., "How PHP Sessions work" or "The DOM in JavaScript").
- **Source Code:** You must write the code **by hand** on your lab sheets.
- **Output:** Paste a screenshot of the output. **Important:** Your code must programmatically print your **Name** and **Roll Number** at the top of every output screen